VARGROUP

Sviluppare un Sistema IA di Caching Semantico e Dashboard FinOps

Estendere le funzionalità del Progetto OpenSource LiteLLM

Le sfide del mercato IA

$200B+
mercato IA globale entro 2025

80%
delle query LLM semanticamente ridondanti

3×
crescita annua costi inferenza vs budget IT

- Le aziende con progetti in produzione faticano a gestire i costi IA
- Le cache tradizionali (1) non riescono ad identificare azioni ripetute in azienda
- Non ci sono ancora sistemi evoluti per fornire viste granulari sui costi (2)
- I costi tipicamente si identificano solo con la fattura finale (2)

Approfondimenti

(1) Cache tradizionale

funzionano come un dizionario, alla richiesta in entrata viene calcolato un hash, e se quell'hash esiste già in memoria, si restituisce la risposta salvata senza chiamare il modello. Il problema è che questo meccanismo è binario o la stringa è identica al 100%, o è un miss. "Qual è il meteo a Milano?" e "Che tempo fa a Milano oggi?" producono hash diversi e generano due chiamate distinte al modello, pagando due volte per la stessa informazione.

(2) Costi AI in azienda

L'industria dell'IA è conosciuta per un modello di business a subscription per seat (tipicamente €20-30/mese per utente) In un contesto enterprise tutto è a consumo quindi si paga a token, dove ogni token è circa ¾ di parola.

Quindi, una singola interazione complessa, con un contesto lungo di sessione, documenti allegati e una risposta articolata può consumare 50.000-100.000 token che se lo si immagina in un'organizzazione con 4.500 persone che interagiscono più volte al giorno si possono trovare sorprese a fine mese.

Il contesto Var Group in parole

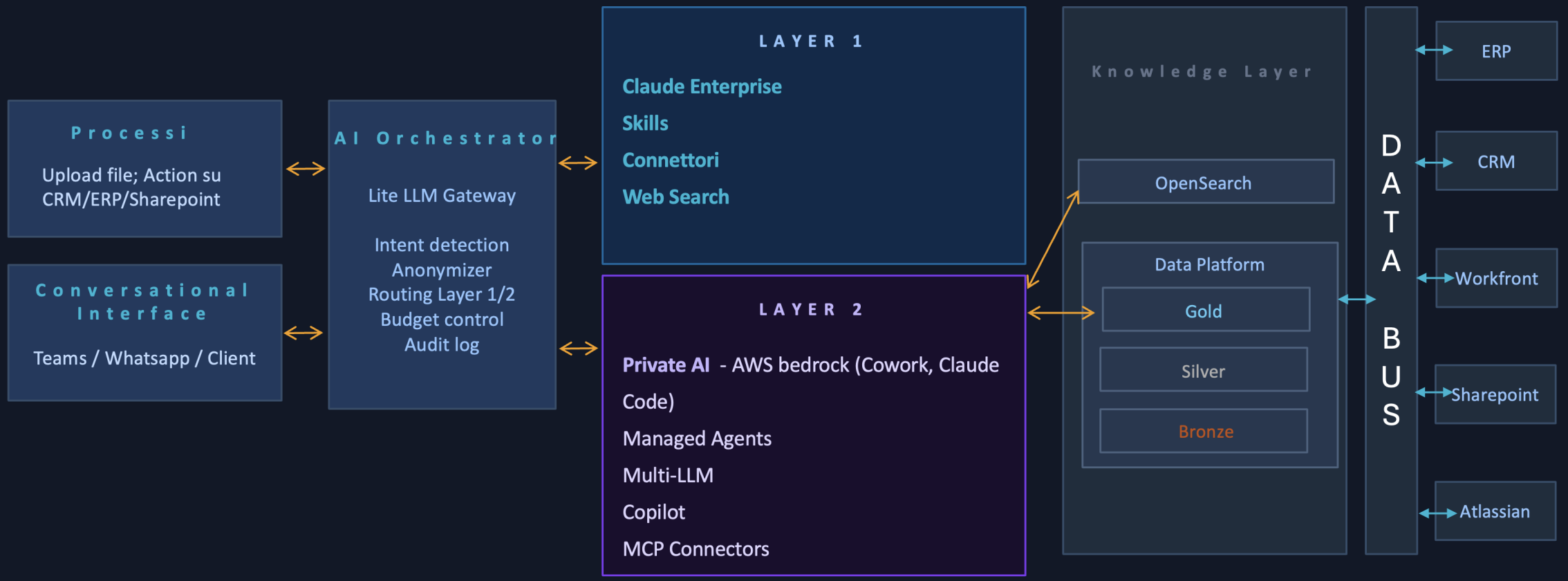
Le aziende oggi operano sotto vincoli normativi stringenti (GDPR, NIS2, AI Act) che impongono controllo preciso su dove i dati vengono elaborati e da chi. In un contesto geopolitico instabile, la sovranità del dato non è più un tema solo legale: è una scelta organizzativa e di architettura.

In Var Group abbiamo risposto con un'architettura AI a due livelli separati.

- **Layer 1** aggrega i modelli pubblici (Claude Enterprise, GPT, Web Search): potenti, aggiornati, ottimi per attività di ricerca, sintesi e navigazione che non toccano informazioni riservate.
- **Layer 2** è un ambiente completamente privato e governato da Var Group su AWS: qui risiedono i modelli proprietari, gli agenti gestiti, i connettori verso i sistemi aziendali (ERP, CRM, SharePoint, Atlassian) e i dati strutturati su tre livelli di sensibilità (Gold, Silver, Bronze).

Il collante tra i due è l'IA Orchestrator che ha il compito di ricevere ogni richiesta (da un processo automatico o da un'interfaccia conversazionale (Teams, WhatsApp, client custom)) e decidere in autonomia dove instradarla. Analizza il contesto della sessione come l'identità dell'utente e la natura del dato coinvolto e se la richiesta è generica e non tocca dati sensibili va su Layer 1 mentre, se elabora documenti riservati o accede a sistemi interni, resta su Layer 2 in ambiente sicuro e compliant.

Il contesto Var Group



LiteLLM: Ruolo dell'Orchestratore e Limiti Riscontrati

COSA È E PERCHÉ LO USIAMO

- Proxy open source per routing multi-provider LLM
- Interfaccia OpenAI-compatibile (drop-in replacement)
- Gestione autenticazione e API key per provider
- Già in produzione su Var Group

LIMITI RISCONTRATI IN PRODUZIONE

Cache esatta only: non riconosce query semanticamente simili ma non identiche

Nessuna dashboard FinOps nativa: zero visibilità aggregata sui costi

Nessun forecast di spesa: impossibile pianificare il budget IA

Multi-tenancy limitata: difficile isolare costi per team e progetto

VARGROUP

Prima degli obiettivi: le basi per capire il progetto

Sei concetti chiave

01 IA generativa

02 App vs API

03 Vincolo GDPR

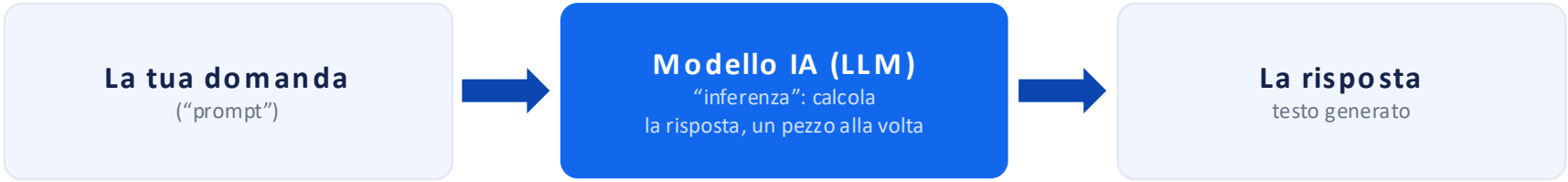
04 Provider gestiti

05 Orchestratore

06 Cache & FinOps

CONCETTO 1 · COME FUNZIONA

Un’IA generativa in breve — e perché ogni domanda ha un costo



Ogni riquadro è fatto di “token”: piccoli mattoncini di testo che il modello legge e scrive.

Si ragiona a token

Il modello non conosce le parole come noi: le spezza in token. Più testo entra ed esce, più lavoro fa.

Si paga a token

Ogni richiesta ha un prezzo. Migliaia di chiamate al giorno = costi che crescono in fretta e vanno governati.

Nessuna memoria tra richieste

Se rifai una domanda equivalente, il modello la ricalcola da capo (e la ripaghi). È qui che serve una cache.

CONCETTO 2 · DUE MODI DI USARE L'IA

L'assistente (app) e l'API: non sono la stessa cosa



L'assistente (app)

- È ChatGPT o Claude che usi da sito o app.
- Una persona scrive in chat e legge la risposta.
- Pensato per l'uso individuale, non per i sistemi aziendali.



L'API (il modello come servizio)

- Il software dell'azienda "chiama" il modello via Internet.
- L'IA viene integrata dentro prodotti e processi.
- È così che le aziende costruiscono soluzioni IA su larga scala.

Il nostro progetto vive nel mondo delle API. È lì che servono controllo dei costi e conformità dei dati — i due temi delle prossime slide.

CONCETTO 3 · IL VINCOLO GDPR

Perché un’Enterprise non può usare ChatGPT/Claude “così com’è”

Non basta che il modello sia bravo: dev’essere conforme TUTTO il flusso dei dati (raccolta → invio → elaborazione → conservazione).

Dove vanno i dati

Con l’app pubblica, prompt e documenti possono uscire dall’UE e finire su server di terzi.

Uso per addestramento

Senza garanzie contrattuali, i dati inviati potrebbero essere riutilizzati per addestrare i modelli.

Dati personali e segreti

Email clienti, contratti, codice: dato personale (GDPR) e know-how aziendale da proteggere.

Controllo e responsabilità

Servono tracciabilità, audit, base giuridica e un responsabile del trattamento: l’app consumer non li offre.

CONCETTO 4 · LA SOLUZIONE… E I SUOI COSTI

La via d’uscita: usare i modelli tramite provider “gestiti”

Cosa risolve

- Usare i modelli di Anthropic e altri provider tramite deployment enterprise conformi (cloud UE / provider gestiti). Ad esempio su AWS o Azure.
- Contratti con garanzie: dati non usati per il training, region UE, riservatezza.
- Il flusso dei dati torna sotto controllo → compatibile con il GDPR perche' tutto in regioni Europee.

Cosa introduce

- Più provider e più modelli da gestire → frammentazione tecnica.
- Costi sparsi e poco visibili: ogni provider fattura a modo suo.
- Nessun controllo unificato: chi spende cosa? quale team? quale progetto?
- I costi si scoprono spesso solo a fine mese, con la fattura.

→ Serve uno strato unico davanti a tutti i modelli, che dia governo e visibilità: l’orchestratore.

CONCETTO 5 · L'ORCHESTRATORE

LiteLLM: un unico "centralino" davanti a tutti i modelli



Cosa fa già bene

- Interfaccia unica, compatibile con lo standard OpenAI.
- Instrada le richieste tra più provider (routing).
- Gestisce chiavi e credenziali. Già in produzione in Var Group.

I due "buchi" che il progetto colma

1. Niente cache semantica —
si ripaga per domande equivalenti.

2. Niente FinOps —
nessuna visibilità, controllo o previsione dei costi.

CONCETTO 6 · I DUE MATTONI DEL PROGETTO

Caching semantico e FinOps

Caching semantico

capire il significato, non le parole esatte

- L'IA trasforma ogni frase in una lista di numeri (embedding) che ne cattura il significato.
- Due frasi "vicine" fra quei numeri = stesso significato (similarità).
- Cache classica: risponde solo se la domanda è identica. Cache semantica: risponde anche se è formulata in modo diverso → meno chiamate, meno costi.

FinOps

governo economico dell'IA

- Attribuire ogni costo a team / progetto / utente.
- Budget + avvisi automatici quando la spesa sfora.
- Previsione (forecast) della spesa, non solo la fattura a posteriori.
- Misurare il risparmio reale prodotto dalla cache semantica.

Obiettivo del Progetto:

Estendere LiteLLM con due componenti originali che colmano i suoi limiti critici in produzione

01 Caching Semantico

Layer di cache basato su embedding e similarity search. Riconosce query semanticamente simili, restituisce la risposta cachata, logga token e latenza risparmiati.

Soglia di similarità configurabile, invalidazione TTL/versioning, persistenza su Redis o vector DB.

02 Dashboard FinOps

Servizio separato che aggrega i consumi per team / progetto / utente, calcola il risparmio reale dalla cache semantica, gestisce budget + alert, e forecasting della spesa mensile.

Aggiornamento near real-time, multi-tenancy, isolamento dati per team.

DELIVERABLE

Output Attesi dal Progetto

01

Architettura & ADR

Diagrammi, Architecture Decision Records, schema data model

02

Documento di Requisiti

Business requirements post design thinking + User Story Mapping

03

Diagrammi UML

Use Cases, diagrammi di sequenza e di componenti

04

MVP Funzionante End-to-End

Proxy + caching semantico + dashboard FinOps integrati

05

Test Automatici (coverage ≥70%)

Unit test + integration test con report di copertura

06

Repository Git + Bug Report

Codice sorgente versionato, documentazione, bug report post-collaudo

Tecnologie consigliate

Backend & Proxy

Python 3.10+
LiteLLM (base)
FastAPI / Flask
Docker + Docker Compose

1

IA Embedding

OpenAI text-embedding-3-small
sentence-transformers (open source)
Cosine similarity (numpy / scikit-learn)

2

Cache & Vector Store

Redis (cache + TTL)
Qdrant o pgvector (vector search)
PostgreSQL (metriche FinOps)

3

Dashboard & Frontend

React + Chart.js
Streamlit (alternativa rapida)
REST API JSON
Git / GitHub o Bitbucket

4

Vincoli del progetto

Processo

- Sessione di design thinking con il committente
- User Story Mapping e PoC di fattibilità tecnica prima dell'implementazione
- Standup periodici con referente Var Group

Architettura

- LiteLLM come proxy base (non reinventarlo)
- Logica originale sostanziale su cache semantica e FinOps
- Design modulare ed estendibile per future funzioni

Qualità

- Test coverage ≥ 70% del codice prodotto
- Separazione test deterministici vs valutazione probabilistica
- Multi-tenancy: dati isolati tra team/progetto

Materiali da condividere

PRIMA DELL'IMPLEMENTAZIONE

- Documento di requisiti di business (post design thinking)
- Diagrammi UML degli Use Cases
- Schema design del database / data model
- Piano di test di unità (target coverage ≥ 70%)
- PoC di fattibilità tecnologica

DOPO IL COLLAUDO (POST-LIVE)

- Bug Reporting (problemi riscontrati in fase di test)
- Codice sorgente su repository Git (GitHub / Bitbucket)
- Documentazione architettura e scelte tecniche (ADR)

Supporto Var Group al team:

Design thinking · Formazione sulle tecnologie · Sessioni di review · Repository Git condiviso

RISORSE

Risorse Utili per il Progetto

Proxy & Routing	
LiteLLM Documentation	docs.litellm.ai
LiteLLM GitHub	github.com/BerriAI/litellm

Embedding & Similarity	
sentence-transformers	sbert.net
OpenAI Embeddings Guide	platform.openai.com/docs/guides/embeddings

Vector Store & Cache	
Qdrant (vector DB)	qdrant.tech/documentation
pgvector (PostgreSQL)	github.com/pgvector/pgvector
Redis Docs	redis.io/docs

Dashboard & Backend	
FastAPI	fastapi.tiangolo.com
Streamlit	streamlit.io
Chart.js	chartjs.org

Per rimanere in contatto

Alessio Borgheresi

Email: alessio.borgheresi@vargroup.com

Stefano Dindo

Email: stefano.dindo@vargroup.com

Laura Ferretti

Email: laura.ferretti@vargroup.com

Michele Massaro

Email: michele.massaro@vargroup.com

Vi chiediamo di inviare una mail con oggetto: **SWE 2026 | <Oggetto della mail>**
Tutte le persone presenti devono essere in copia alla mail

Note organizzative

Calendario disponibilità

Attività	Durata	Modalità	Data proposta	Note
Sessione di design thinking	3 ore	Presenza	Da definire	
Follow-up raccolta requisiti (follow-up)	1 ora	Teams	Da definire	Chiarimenti post elaborazione output design thinking
Formazione LiteLLM	2 ore	Teams	Da definire	Video + hands-on
Formazione Embedding & Caching	2 ore	Teams	Da definire	Video + hands-on
Formazione FinOps & Dashboard	1,5 ore	Teams	Da definire	Video + hands-on
Stand-up di allineamento	30mn	Teams	Da definire	
Q&A Zone	1 ora	Teams	Da definire ogni 2 settimane	



VARGROUP